UNIVERSITY OF ESWATINI

Department of Computer Science

Practices in Software Engineering I

Submitted to Dr. E. L Dube, Lecturer

In partial fulfilment of the requirements for CSC392: Practices in Software Engineering I

Group members:

1. Nompumelelo Sithole_202201600
2. Nokwazi Ginindza_202203354
3. Philangezwi Ndlela_ 202203013
4. Bandile Mvelase_202203849

Project name: Digital Gym Management System

1. Introduction

At the University of Eswatini (UNESWA), the gym serves a diverse group of users, including staff members, students, pensioners, and external clients such as SNPF members and those on Stop Order arrangements. Currently, the gym relies on a manual filing system to manage member information and categorize payments, with physical folders used to sort data by user group. While this method works, it comes with several challenges it's time-consuming, prone to human error, and not scalable as membership grows. Finding specific information quickly can be difficult, and maintaining accurate, up-to-date records across different categories requires significant administrative effort.

To address these issues and improve overall efficiency, this project proposes the development of a Digital Gym Management System. The new system will streamline operations by handling member registration, organizing users by category, tracking payments, managing bookings, and generating reports all in a secure, centralized platform. This shift from manual to digital processes will not only enhance day-to-day operations but also ensure better accuracy, faster access to data, and greater flexibility for the gym to evolve and expand its services in the future.

2. Client/User Involved

1. Information to be Provided by the User

1. During Registration:

   - Personal Details:
     • Full name, surname, date of birth, gender, PIN.
     • Postal and physical addresses, email, mobile/landline numbers.
     • Occupation and employer (for categories like Company Contract Employee).
   - Membership Category:
     • Proof of eligibility such as student ID for "UNESWA Student", staff card for "UNESWA Staff", national ID for "Senior Citizen".
     • Institution name (e.g., "UNESWA (K/L/M Campus)" for students/staff).
   - Activities of Interest:
     • Selection of preferred activities (Swimming, Aerobics, Weights, Other).
   - Emergency/Health Information:
     • Declaration of physical fitness and lack of medical conditions.

2. During Payment:

   - Payment Details:
     • Method (Cash, Stop Order, SNPF, etc.).

• Payment verification such as bank details for stop orders, receipts for cash payments.

3. During Facility Use:

- Compliance Acknowledgment:
  • Agreement to adhere to rules including no smoking, pre-swim showers, towel usage.

2. Services to be Provided by the User

4. During Registration:

- Waiver Signing:
  • Electronic signature accepting liability for injuries/risks.
- Rule Acknowledgment:
  • Confirmation of understanding and compliance with gym/pool rules such as no running, mandatory swimming caps.

5. During Facility Use:

- Booking Reservations:
  • Advance booking of classes, swimming slots, or equipment.
- Incident Reporting:
  • Reporting rule violations or unsafe conditions to staff.

3. Resources to be Provided by the User

6. During Facility Use:

- Hygiene and Safety:
  • Towel for personal hygiene (required during every visit).
  • Swimming cap for pool access.
- Locker Security:
  • Personal padlock for lockers (as specified in the rules).
- Appropriate Attire:
  • Activity-specific clothing (e.g., swimwear for the pool, gym attire for weights).

4. Facilities Provided by the Gym

- Exercise Areas:
  • Gymnasium (weights, aerobics equipment).
  • Swimming pool (with showers and change rooms).
- Support Services:
  • Instructor-led sessions.
  • Secure lockers (though users must bring their own padlocks).
- Administrative Support:
  • Membership management (code assignment, renewal reminders).
  • Rule enforcement and staff assistance.

II. Timeline for Provision

7. 1. Registration Phase:

   • Personal information, eligibility proof, and waiver signing.
   • Payment details and activity selection.

8. 2. Pre-Visit Phase:

   • Booking reservations for classes/pool slots.

9. 3. During Visit:

   • Compliance with rules (e.g., no smoking, pre-swim shower).
   • Provision of resources (towel, padlock, appropriate attire).

10. 4. Post-Visit:

   • Returning lockers empty and unlocked (as per rules).
   • Renewing memberships or updating preferences as needed.

III. Key Compliance Requirements

- Rules:
  • Users must adhere to all facility rules such as no nudity in the pool, no alcohol/drugs during every visit.
- Validation:
  • Staff may verify membership status, IDs, or compliance with rules at entry or during random checks.

3. Risks

Potential risks have to be identified as soon as possible and appropriate strategies must be devised to deal with these risks

During software development there is a possibility of encountering certain risks. The potential risks that can be encountered during software development and the strategies to deal with them include:

I. Requirements Risks – this may occur due to misunderstanding, incomplete or changing requirements. The consequences of this risk are that developers may build the wrong functionality software which may cause project delays hence higher rework cost and conflicts with stakeholders.

Strategy: we need to conduct thorough interviews with stakeholders to ensure we understand exactly what they require in or for the software. We also need to keep an open mind for any changes that may be needed mid-project. Another way to mitigate the risk is by applying a requirement traceability matrix which we will use to tally every step we make in the software with the requirements stated.

II. Technical Risks – these are risks that emerge from the technology, system architecture and tools used in the development process. They can be caused by choosing technologies that developers are unfamiliar with, using incompatible tools and systems as well as a system architecture that may not handle expected loads. Failure to counter these risks may lead to the software becoming difficult to manage, it crashing or being exposed to vulnerabilities.

Strategy: conduct technical feasibility studies before development begins and build prototypes. We can also engage experienced architects to continuously review and test the architecture of the software we are developing.

III. Resource Risks – these are risks related to the availability, capability and stability of the team members and resources required for the project. They may occur due to key personnel leaving mid project, insufficient staff and funds to meet deadlines. Also, if the team lacks the required skillset to complete the project, it stands at a risk. The result of these risks include delays, poor quality work and the project becomes unsustainable.

Strategy: we need to have a resource management plan and maintain a skill matrix and we also need to train team members for the skills required to complete the project. We can also motivate our team with incentives and feedback for every task completed so that they enjoy their work and make sure to do it diligently at all times for efficiency and effectiveness of the project.

IV. Quality Risks – risks that the software will not meet the quality expectation of users. The arise from inadequate testing, rushed development cycles and lack of testing and code reviews. These risks can lead to poor software performance, negative user experience and high costs of fixing issues after the software has been deployed.

Strategy: we can mitigate these risks by developing a quality assurance plan at the beginning of the project and ensure that we regularly engage with our users in the testing phases of the software. We can also implement automated and manual testing strategies and perform code reviews at every increment.

V. User Adoption Risk – this is a risk that affects gym staff members who may not understand, accept or effectively use the software due to insufficient or ineffective training. The result of this risk is that it may cause frustration amongst staff members which will then lower efficiency, reliance on manual processes or they may even abandon the system entirely.

Strategy: we need to offer role-based training sessions for the staff members of the gym for example the front desk person on how to operate the system. We could also provide learning manuals or tutorials to help the staff members navigate and use the software effectively.

4. Standards, Guidelines, and Procedures

1. Standards
   Technical standards:

- Data Security:

  • Encrypt sensitive data including PINs and payment details using AES-256 or equivalent.
  • Comply with GDPR/regional data protection laws for user privacy.

- System Architecture:

  • Use RESTful APIs for backend communication.
  • Adopt cloud-based hosting (AWS, Azure) for scalability.
  • Ensure cross-platform compatibility (web, iOS, Android).

- Accessibility:

  • Follow WCAG 2.1 guidelines for UI/UX (like screen reader support, contrast ratios).

Operational standards:

- Membership Validation:

  • Verify eligibility such as student/staff IDs, national IDs during registration.
  • Auto-generate unique membership codes.

- Compliance:

  • Enforce pool/gym rules programmatically such as block entry without a swimming cap.

II. Guidelines

Development Guidelines:

- Code Quality:

  • Follow modular programming practices (MVC architecture).
  • Use version control (Git) with detailed commit messages.

- Documentation:

  • Maintain API documentation (Swagger/Postman).
  • Create user manuals for staff and members.

- Testing:

  • Perform unit, integration, and UAT (User Acceptance Testing).
  • Validate edge cases like double bookings, expired memberships.

User Experience (UX) Guidelines:

- Registration Portal:

- Simplify form fields such as auto-fill postal/physical addresses.
- Include tooltips for PIN/eligibility requirements.

- Mobile App:

  - Push notifications for booking confirmations, rule reminders, and renewals.
  - Offline access to membership cards and schedules.

Compliance Guidelines:

- Waiver Management:

  - Ensure digital signatures are legally binding like DocuSign integration.
  - Archive signed disclaimers securely.

- Rule Enforcement:

  - Display rules prominently during booking such as Pre-swim shower required

Procedures

For Developers:

1. Data Migration:
   - Digitize existing member records into a SQL/NoSQL database.
   - Validate data integrity.

2. Feature Implementation:

   - Prioritize core modules: registration, payment, booking, and waiver systems.
   - Integrate QR/RFID check-in using libraries

3. Deployment:

   - Use CI/CD pipelines such as GitHub Actions for updates.
   - Conduct load testing before full launch.

For Administrators/Staff:

1. User Onboarding:

   - Train staff to use the admin dashboard like manage memberships, view violations.
   - Provide members with tutorials for app/portal usage.

2. Incident Management:

   - Log and escalate violations such as non-compliance with pool rules via the dashboard.
   - Update rules dynamically.

3. Maintenance:

   - Schedule monthly audits for data accuracy and security.

• Renew SSL certificates and apply security patches.

For Users:

1. Registration:

• Provide valid IDs and PINs during signup.
• Sign the digital disclaimer before accessing facilities.

2. Facility Use:

• Book sessions in advance.
• Bring required resources (towel, padlock, swimming cap) for each visit.

3. Compliance:

• Follow all posted rules such as no smoking, no nudity in pools.
• Report issues via the app/portal (e.g., broken equipment, rule violations).

IV. Roles and Responsibilities

Role	Key Responsibilities
Developers	Ensure code quality, data security, and integration with existing systems.
Project Manager	Oversee timelines, coordinate testing, and manage stakeholder communication.
Admin/Staff	Validate memberships, enforce rules, and handle incident reports.
Users	Provide accurate information, comply with rules, and use resources responsibly.

V. Compliance Timeline

Pre-Launch:

• Finalize legal approvals for digital waivers (1–2 weeks).
• Complete UAT and staff training (2–3 weeks).

Post-Launch:

• Monitor system performance and user feedback (ongoing).
• Update rules/features quarterly based on operational needs.

5. Organization of the Project

The project is structured as follows:

- **Project Manager**: Oversees the entire process, manages timelines, and ensures deliverables meet quality standards.

- **Analyst**: Collects and refines requirements, ensuring alignment with stakeholder needs.
- **Design Team**: Focuses on creating the system architecture, UI/UX design, and database schema.
- **Development Team**: Implements functionality, coding the back-end and front-end.
- **Testing Team**: Ensures the application performs as intended and meets all quality benchmarks.

Training sessions will be scheduled for personnel involved to familiarize them with tools and techniques specific to this project. Collaboration and clear communication protocols will be established to ensure smooth execution.

6. Plan for Developing Gym Management Software Using the Agile Model

Why is Agile?

Agile is a way of working where we will build the software in small steps, called "sprints." Each sprint lasts 2 weeks, and at the end, we have a working piece of the software that the gym can test and give feedback on. This lets us make changes as we go, ensuring the final product fits the gym's needs.

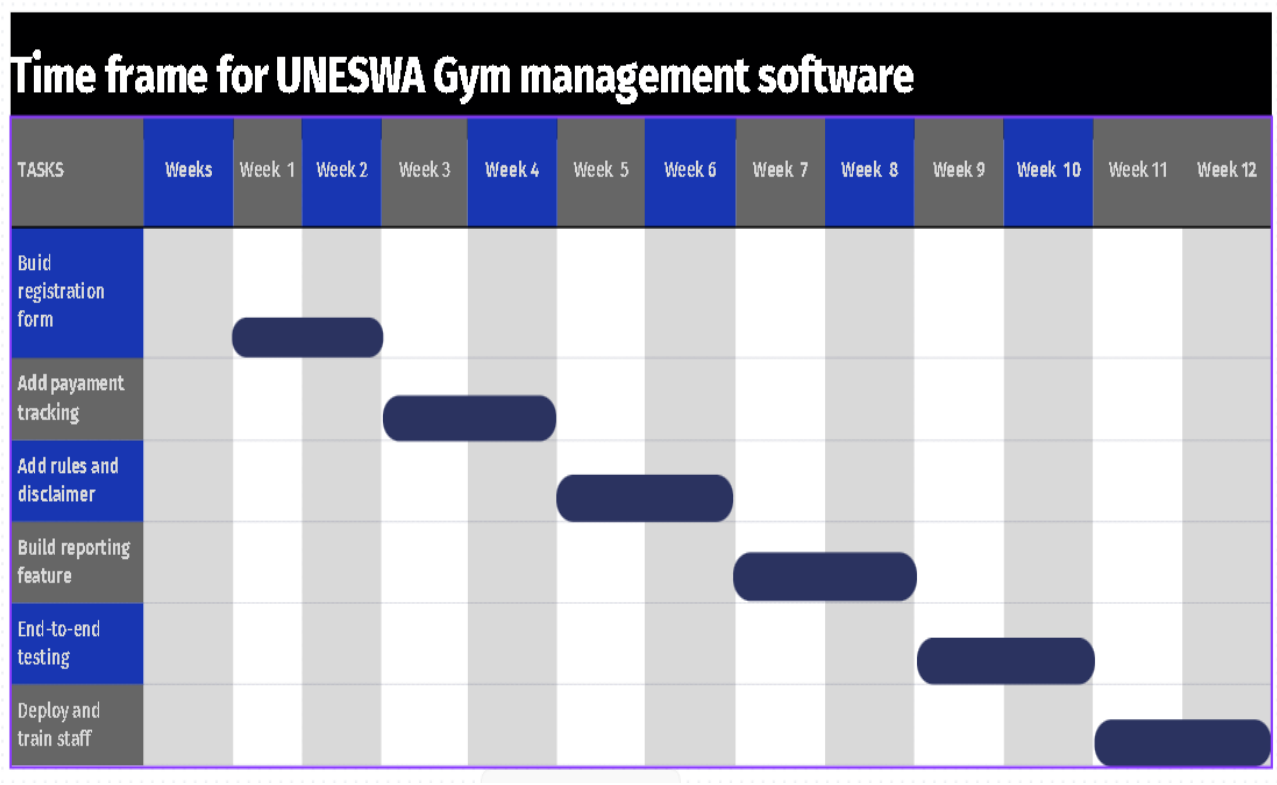
Project Phases (Breaking the Work into Sprints)

We willl work in 2-week sprints, aiming to finish the core software in 6 sprints (12 weeks total). If the gym wants more features later, we can add more sprints. Here is our the plan

Sprint	What We'll Do	What the Gym Gets	How We Check It's Done
Sprint 1 (Weeks 1-2)	Talk to gym staff to understand their needs (like membership forms and payment categories. Set up the database to store member information. Build a simple form to add new members.	A form to register members (e.g., name, category, contact info).	Gym staff can add a member, and see their details saved.
Sprint 2 (Weeks 3-4)	Add a way to categorize members (e.g., Students, Staff,	A payment tracking feature	Staff can assign a category (e.g., "UNESWA

	general members) and track their payments. Create a screen to show payment details		Student”) and record a payment.
Sprint 3 (Weeks 5-6)	Add the gym and swimming pool rules to the system. Create a digital form for members to sign the disclaimer	Rules and disclaimer section	Members can read rules (e.g., “Not run”) and sign the disclaimer digitally.
Sprint 4 (Weeks 7-8)	Build a report feature to show member lists and payment status.	Reports the staff can view.	Staff can see a list of all members and their payment status (e.g., who paid for March).
Sprint 5 (Weeks 9-10	Test the whole system to make sure everything works together. Fix any problems the gym staff find	A complete, working system.	The system passes all tests, and staff confirm it works for their needs
Sprint 6 (Weeks 11-12	Put the software on a cloud server so the gym can use it. Train the staff on how to use it. Move the paper records (like the member list) into the system	A live system with trained staff	Staff can use the system for a week without issues.

Gantt chart: representing tasks and time required to complete them



7. Requirements Analysis and Design

- **Methods and Techniques**:
  - Requirements analysis will be conducted through stakeholder interviews, surveys, and workshops.
  - Agile methodologies will be employed to adapt to evolving requirements.
  - Unified Modeling Language (UML) diagrams will be used to visualize system components.
- **Resources and Tools**:
  - Tools such as Microsoft Visio for design diagrams and Jira for task management.
  - A relational database management system (e.g., MySQL) for data handling.
- **Design Goals**:
  - Create a scalable system capable of supporting multiple gyms and their members.
  - Ensure the design meets usability standards, including responsive interfaces for both mobile and desktop devices.
  - Incorporate robust security measures such as encryption for sensitive data.

8. Implementation

What We Need to Build the Software

- **Computers and Servers**:
  - Laptops for the team.
  - AWS cloud server to host the software.
- **Tools**:
  - Visual Studio Code (for coding).
  - MySQL (to store data).
  - Git and GitHub:
    - GitHub repository for code storage.
    - GitHub Issues for task tracking.
    - GitHub Pull Requests for code reviews.
    - GitHub Actions for automated testing/deployment.
  - Tesseract (to scan paper records).
- **People**:
  - 1developers (code and manage GitHub).
  - 1 designer (create UI, commit to GitHub).
  - 1 project manager (oversee GitHub Issues/Milestones).
  - 1 architect
  - Temporary staff (migrate records, tracked in GitHub).

- **Training**:
  - User manual and videos in GitHub Wiki.
  - Project manager trains staff in Sprint 6 (feedback in GitHub Issues).

What the Software Will Do

- **Membership Management**: Form to add members (name, category, contact).
- **Payment Tracking**: Assign categories (e.g., Students, Staff) and record payments.
- **Rules and Disclaimer**: Show gym/pool rules and allow digital disclaimer signing.
- **Reports**: Show member lists and payment status.

How We'll Build It

- Build in small pieces, one per sprint (6 sprints total).
- Use GitHub:
  - Store code in a repository; create branches per feature (e.g., feature/registration-form).
  - Commit changes, review via Pull Requests, merge to main branch.
  - Automate testing/deployment with GitHub Actions.

- In Sprint 6, migrate paper records using Tesseract and manual entry, tracked in GitHub Issues.

9. Testing (Making Sure the Software Works)

Where We'll Test

- On our laptops (local testing of features).
- On a staging server (AWS, mimics real setup, deployed via GitHub Actions).

What We'll Use for Testing

- Laptops for the team.
- Tools: Selenium (screens), Postman (backend), JUnit (code), GitHub Actions (automated testing).

How We'll Test

- **Each Sprint**:
  - Test the new feature (e.g., registration form in Sprint 1).
  - Run automated tests with GitHub Actions (JUnit, Selenium).
  - Log bugs in GitHub Issues; staff feedback logged in GitHub (e.g., "Add emergency contact").
- **Big Test (Sprint 5)**:
  - Test everything together (register, categorize, pay, report).
  - Log/fix bugs in GitHub Issues; retest via GitHub Actions.
- **Final Check**:
  - Staff confirm system works (feedback in GitHub Issues).

Testing Steps

- Write test cases (e.g., "Can a member sign the disclaimer?") in GitHub repo (tests folder).
- Run tests locally and via GitHub Actions; log failures in GitHub Issues.
- Fix issues, commit, retest until all pass.
- Staff do final test in Sprint 5 (results in GitHub).

10. Resources

Digital Gym System Project: Tools and Requirements

1. Hardware Requirements

Core Infrastructure

- Servers:
  - Cloud Hosting: Use AWS, Azure, or Google Cloud for scalable storage, databases, and backend services.
  - On-Premise Servers: Physical servers (e.g., Dell PowerEdge) if opting for local hosting (requires RAID configurations for redundancy).
- Networking:
  - Routers, switches, and high-speed Wi-Fi access points for seamless connectivity.
  - Firewalls (e.g., Cisco ASA) to protect against cyber threats.
- Backup Systems:
  - External hard drives or NAS (Network-Attached Storage) for local backups.
  - Cloud backup solutions (e.g., AWS S3) for disaster recovery.

User-Facing Devices

- Check-In Systems:
  - Tablets or kiosks with QR code/RFID scanners (e.g., Zebra TC52) at gym entry points.
  - Biometric scanners (optional) for enhanced security.
- Staff Devices:
  - Computers/laptops for admin tasks (e.g., HP EliteBook, Dell Latitude).
  - Printers for issuing temporary passes or receipts (e.g., HP LaserJet).
- Member Devices:
  - Mobile devices (iOS/Android) for app usage.
  - Wearables (e.g., Fitbit) for optional activity tracking integration.

Security Hardware

- CCTV cameras (e.g., Hikvision) for facility monitoring.
- Access control systems (e.g., RFID door locks for restricted areas).
- UPS (Uninterruptible Power Supply) to prevent downtime during outages.

2. Software Requirements

Development Tools

- Frontend: HTML
- Backend: Node.js
- Database: MySQL for member data storage.
- APIs: RESTful APIs for integration with payment gateways - Version Control: Git with platforms like GitHub or GitLab.
- Testing Tools: Selenium (automation), Postman (API testing), Jira for bug tracking.

Administrative Tools

- Dashboard: Custom admin panel (e.g., built with Angular or React) for managing memberships, bookings, and violations.

- Analytics: Tools like Tableau or Google Analytics for usage reports.
- E-Signature: DocuSign or Adobe Sign for digital waivers.

Security Software

- Encryption tools- OpenSSL for data protection.
- Antivirus software- Norton for endpoint security.
- SIEM (Security Information and Event Management)- Splunk for threat detection.

3. Other Tools and Resources

Third-Party Services

- Payment Gateways: Integrate with local banks, SNPF, or stop-order systems.
- Cloud Services: AWS Lambda for serverless functions, Firebase for real-time databases.
- Communication Tools: Slack/Microsoft Teams for team coordination.

Training and Documentation

- Training modules (e.g., video tutorials, PDF guides) for staff and users.
- LMS (Learning Management System) for onboarding (e.g., Moodle).

Physical Infrastructure

- Office space for IT/admin teams.
- Power backups (generators/inverters) for uninterrupted operations.

4. Scalability Considerations

- Modular Architecture: Design systems to add features (e.g., new activity types) without overhauling existing code.
- Load Balancers: Deploy tools like NGINX to handle high traffic during peak hours.
- Edge Computing: Use CDNs (e.g., Cloudflare) for faster app performance globally.

5. Cost-Efficiency Tips

- Start with a Minimum Viable Product (MVP) using open-source tools PostgreSQL.
- Use hybrid cloud models to balance cost and performance.
- Lease hardware RFID scanners instead of purchasing outright.

6. Compliance and Legal Tools

- GDPR/Data Protection Compliance: OneTrust for privacy management.
- Audit Logging: Implement AuditD for tracking system changes.

11. Quality assurance

To ensure that the software meets quality requirements, organisations typically follow structured procedures and standards. These procedures align with industry best practices and internal quality goals. Procedures used to assure that the software being developed meet the quality requirements stated are:

1. Software Development Lifecycle – this is a structured process used by developers to design, test and deploy a software. It ensures that software is built systematically and meets both functional and quality requirements. The structured approach prevents ad-hoc development which often leads to poor quality, continuous testing detects issues early making them easier and cheaper to fix. The SDL can also ensure quality through documentation which maintains clarity and traceability throughout the project and stakeholder involvement whereby the final product aligns with user expectations.

II. Quality Management Plan – it refers to a systematic process of defining, documenting and implementing quality standards and procedures to ensure that a software product meets its specified requirements and satisfies stakeholder expectations. The Quality Management Plan ensures software quality by preventing defects rather than reacting to them also by adhering to predefined standards and guidelines (ensuring compliance), the software maintains consistency, reliability and regulatory compliance. It also improves customer satisfaction by delivering a high-quality product that performs reliably and meets user expectations contributing to overall user trust and satisfaction.

III. Risk Management Plan – it refers to a structured approach to identifying, analysing and responding to potential risks that may affect a project. It provides guidelines for assessing uncertainties, minimising threats and maximising opportunities to ensure the project stays on track. Risk Management Plan ensures quality assurance by identifying risks early before they become problems, focusing on the most critical risks that could impact the software functionality, usability or security. Ensuring compliance also maintains high quality software output by aligning with industry standards and best practices.

IV. Requirement Management Procedures – these are structured processes used to define, document, track and validate software requirements throughout the development lifecycle. These procedures ensure that the software meets quality requirements by maintaining clarity, consistency and alignment with stakeholder's needs. They do so by gathering and analysing stakeholder needs to ensure all necessary features and functionalities are documented also by clearly defining and organising requirements to avoid ambiguity, making the understandable for developers, testers and business teams.

12. Changes

Change is inevitable in all large software projects. The system requirements change as the system responds to external pressures, competition and changed management priorities. As new technologies become available, new approaches to design and implementations become possible. Therefore, it is essential that whatever software model is used can accommodate changes to the software being developed. Clear procedures put in place to deal with these changes include:

I. Change anticipation – this is where the software process includes activities that can anticipate or predict possible changes before significant rework is required. A prototype system may be developed to show some key features of the system to customers for example.

They can experiment with the prototype and refine the requirements before committing to high software production costs.

II. Change tolerance – this is where the process and the software are designed so that changes can be easily made to the system. Proposed changes may be implemented in increments that have not yet been developed and this normally involves some form of incremental development. If these changes are impossible, then only one single increment may have to be altered to incorporate the change.

III. Change identification – this process in software development ensures that the team can manage modifications effectively without disrupting project goals or quality. It helps track, asses and implement changes systematically.